

Gestion de mon projet :

- Technologies choisies :

- Python
- Django
- Css
- JavaScript
- Figma
- Drow.io

- Extensions :

- Let's Encrypt
- BeautifulSoup4
- Psycpg2

Pourquoi ces choix ?

Pourquoi le langage Python ?

J'ai choisi le langage Python afin d'apprendre un langage orienté objet et, sur le long terme, éventuellement me spécialiser dans le domaine de l'intelligence artificielle.

Python, étant un langage POO, me permet d'en comprendre les bases, de commencer à le manipuler et à le maîtriser progressivement.

Ne connaissant aucun langage auparavant, Python me permet d'entrer dans le monde du développement avec une syntaxe relativement simple à comprendre. La seule difficulté notable est le respect strict de l'indentation.

Pour ma part, je trouve ce langage accessible, mais il n'est pas pour autant simple à maîtriser. En effet, bien que la syntaxe soit facile à apprendre, il est plus complexe de savoir où et quand placer certaines lignes de code pour respecter la logique du programme.

Pourquoi Django ?

J'ai choisi le framework Django pour sa polyvalence. Il est basé sur Python, puissant et modulaire.

Django inclut de nombreuses fonctionnalités natives, contrairement à Flask, telles que l'architecture **MVT** (Modèle-Vue-Gabarit), la gestion des modèles avec **User**, un système d'authentification intégré et divers mécanismes de sécurité. Tout cela permet un gain de temps considérable.

Toutefois, cette abstraction peut aussi être une faiblesse pour un apprenant, car certaines parties de Python et SQL sont gérées automatiquement par Django. Cela limite la compréhension de ce qui se passe en arrière-plan, mais représente un avantage pour les développeurs expérimentés.

Pourquoi Css ?

J'ai choisi Css pour la mise en page du site, je l'utilise pour démarrer dans le design, ne maîtrisant aucun framework de mise en page comme Bootstrap ou Tailwind, cependant à l'avenir ou en cours de projet, j'essayerai Bootstrap étant installé mais pas utilisé.

Pourquoi JavaScript ?

JavaScript est un langage extrêmement polyvalent qui permet de faire presque tout.

Pourquoi ne pas l'avoir choisi dès le départ alors ? Comme expliqué plus haut, mon objectif final n'est pas de devenir développeur web, mais de me spécialiser éventuellement en IA. **JavaScript n'était donc pas indispensable au départ.**

Cependant, Python seul ne permet pas d'apporter du dynamisme à une application web. Pour remédier à cela, JavaScript devient nécessaire.

Par exemple, pour pré-remplir un formulaire avec les données d'un utilisateur sans recharger la page ou pour valider une action sur un bouton sans attendre un rechargement complet, **JavaScript apporte une meilleure expérience utilisateur.**

Il permet également d'interagir avec d'autres langages, rendant l'interface plus fluide et naturelle.

Actuellement, je ne connais rien à JavaScript, mais j'espère, d'ici la fin du projet, rendre mon application dynamique aux endroits où c'est nécessaire.

Pourquoi Figma :

J'ai utilisé Figma, un logiciel de modélisation de maquettes, pour créer des mockups et des wireframes. Cela m'a permis de réfléchir au design et à l'agencement visuel potentiel de mon projet avant son développement.

Pourquoi Draw.io :

J'ai utilisé Draw.io, un outil de création de diagrammes, pour concevoir des schémas de base de données et des diagrammes d'architecture. Cela m'a permis de mieux organiser la structure du projet et de visualiser les interactions entre les différents composants de l'application.

Grâce à lui, j'ai pu réaliser :

- Un **Use Case**, schématisant les fonctionnalités accessibles selon le rôle de l'utilisateur.
- Un ou plusieurs **diagrammes de séquence**, expliquant les réactions du site face à certaines actions de l'utilisateur.
- Un **diagramme de classes**, montrant comment j'ai visualisé les interactions entre les différents modèles.

Avec ces diagrammes, j'ai pu organiser mon code de manière plus structurée, ce qui m'a évité des pertes de temps dues à d'éventuels allers-retours sur certaines fonctionnalités.

Les extensions :

Pourquoi ai-je eu recours à des extensions ?

Les extensions installées sont des **bibliothèques** qui ajoutent des fonctionnalités que Python et Django ne possèdent pas nativement ou qui simplifient certaines tâches complexes.

Elles permettent de réduire la difficulté du développement en automatisant certaines actions essentielles au bon fonctionnement de l'application.

Let's Encrypt, qu'est-ce que c'est ?

Let's Encrypt est une **autorité de certification (CA – Certificate Authority)** qui permet de **générer gratuitement des certificats SSL** pour utiliser le protocole **HTTPS**.

Cela permet :

- de **chiffrer les données** échangées sur le site,
- d'**améliorer le SEO** (référencement) sur Google,
- d'**éviter les alertes de sécurité** comme "*Site non sécurisé*".

Grâce à Let's Encrypt, j'ai pu générer un certificat SSL pour mon **nom de domaine**, sécurisant ainsi mon application.

BeautifulSoup4, à quoi ça sert ?

BeautifulSoup4 est une bibliothèque Python qui **permet de manipuler du code HTML** facilement.

Elle permet notamment :

- de **récupérer et analyser** des pages web,
- de **nettoyer du code HTML**,
- de **naviguer** dans des balises HTML,
- de **convertir** du code HTML en texte lisible.

J'ai utilisé cette extension pour **la modération des commentaires par e-mail** dans mon application. Elle m'a permis de récupérer plus facilement des **IDs de trajets, des commentaires ou des utilisateurs**, sans avoir à **manipuler le HTML manuellement** avec du traitement de texte classique.

Au lieu d'écrire du code complexe en Python pour supprimer les balises HTML et découper du texte, **BeautifulSoup4** me permet de le faire en quelques lignes de code, rendant mon application plus efficace.

Psycopg2, à quoi ça sert ?

Psycopg2 est une bibliothèque Python permettant de **connecter une base de données PostgreSQL** à un framework comme **Django**.

Son rôle est d'exécuter des requêtes SQL (**SELECT, INSERT, UPDATE, DELETE...**) en arrière-plan.

Pour dire simplement :

Psycopg2 sert de "**traducteur**" entre **Django ORM et PostgreSQL**. Il permet à Django d'envoyer

des requêtes en Python et de les convertir automatiquement en SQL, compréhensible par PostgreSQL.

Grâce à **Psycopg2**, Django peut :

- **interagir avec la base de données,**
 - **exécuter des requêtes,**
 - **gérer les migrations** de manière transparente via la console.
-

En conclusion :

Ce projet me permet d'apprendre et de progresser à travers différentes technologies.

Django et Python me permettent d'acquérir de solides bases en développement web, du fonctionnement de la logique de codage, tandis que Bootstrap et JavaScript améliorent l'aspect visuel et l'interactivité de l'application au vue de l'utilisation que j'en fait (JavaScript).

Grâce aux **extensions**, je peux automatiser des tâches et rendre mon développement plus efficace.